

FK

SQL Foreign Key

Poora Guide — Beginners se Advanced

Syntax • Examples • Constraints • Best Practices

CREATE TABLE

ALTER TABLE

ON DELETE CASCADE

JOIN Queries

Self-Referencing

1. Foreign Key Kya Hota Hai?

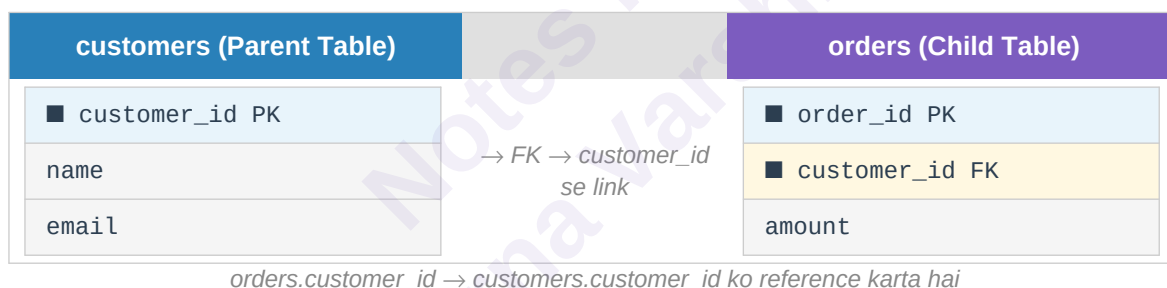
Foreign Key (FK) ek column ya columns ka group hota hai jo ek table ko doosri table se connect karta hai. Ye relational databases ka sabse important concept hai. FK ye ensure karta hai ki ek table mein sirf wahi values aa sakti hain jo doosri table mein already exist karti hain.

■ **Simple Definition:** Foreign Key ek 'link' hota hai do tables ke beech. Jis table mein FK hota hai use **Child Table** kehte hain, aur jis table se reference liya jaata hai use **Parent Table** kehte hain.

Kyun zaruri hai?

- **Referential Integrity:** Galat data insert hone se bachata hai
- **Data Consistency:** Tables ke beech relationships consistent rehte hain
- **Database Design:** Real-world relationships ko model karna aasaan ho jaata hai
- **JOIN Queries:** Tables ko efficiently join karne mein madad karta hai

Parent–Child Table Relationship:



2. Basic Syntax

Method 1 — CONSTRAINT naam ke saath (Recommended)

```
-- Pehle parent table banao
CREATE TABLE customers (
  customer_id INT PRIMARY KEY,
  name        VARCHAR(100),
  email       VARCHAR(100)
);

-- Phir child table banao FK ke saath
CREATE TABLE orders (
  order_id    INT PRIMARY KEY,
  customer_id INT NOT NULL,
  amount      DECIMAL(10,2),

  CONSTRAINT fk_customer      -- constraint ka naam
  FOREIGN KEY (customer_id) -- child column
  REFERENCES customers(customer_id) -- parent column
);
```

Method 2 — Inline (Short way)

```
CREATE TABLE orders (
  order_id    INT PRIMARY KEY,
  customer_id INT REFERENCES customers(customer_id),
  amount      DECIMAL(10,2)
);
```

■ **Yaad rakho:** Parent table PEHLE banana zaroori hai, kyunki child table parent ko reference karta hai. Ulta karne par error aayega!

3. ALTER TABLE — Existing Table mein FK Add Karna

Agar table pehle se bana hua hai aur baad mein Foreign Key add karni ho, toh ALTER TABLE use karte hain.

FK Add Karna

```
ALTER TABLE orders
ADD CONSTRAINT fk_customer_id
  FOREIGN KEY (customer_id)
  REFERENCES customers(customer_id)
  ON DELETE CASCADE;
```

FK Remove Karna

```
-- MySQL mein:  
ALTER TABLE orders DROP FOREIGN KEY fk_customer_id;  
  
-- PostgreSQL mein:  
ALTER TABLE orders DROP CONSTRAINT fk_customer_id;
```

Multiple FK — Ek Table mein Kai Foreign Keys

```
CREATE TABLE order_items (  
  item_id    INT PRIMARY KEY,  
  order_id   INT,           -- orders table se link  
  product_id INT,           -- products table se link  
  quantity   INT,  
  
  FOREIGN KEY (order_id) REFERENCES orders(order_id),  
  FOREIGN KEY (product_id) REFERENCES products(product_id)  
);
```

Notes by
Aparna Varshney

4. ON DELETE / ON UPDATE Options

Jab parent table ki koi row delete ya update ho, toh child table ke related rows ka kya hoga — yeh hum ON DELETE aur ON UPDATE se control karte hain.

Option	Kya hota hai?	Kab use karein?
CASCADE	Parent delete/update → child bhi delete/update	Blog posts ke saath comments
RESTRICT	Child exist kare toh parent delete/update BLOCKED	Account delete se pehle cleanup
SET NULL	Parent delete → child ka FK = NULL	Manager resign kare toh employees
SET DEFAULT	Parent delete → child ka FK = default value	Rare, specific cases mein
NO ACTION	RESTRICT jaisa, check deferred hota hai	PostgreSQL transactions mein

Sabke Code Examples:

```
-- 1. CASCADE: parent delete hone par child bhi delete
FOREIGN KEY (customer_id) REFERENCES customers(customer_id)
ON DELETE CASCADE ON UPDATE CASCADE;

-- 2. SET NULL: parent delete hone par NULL ho jaata hai
FOREIGN KEY (manager_id) REFERENCES employees(emp_id)
ON DELETE SET NULL;

-- 3. RESTRICT: parent tabhi delete hoga jab child na ho
FOREIGN KEY (dept_id) REFERENCES departments(dept_id)
ON DELETE RESTRICT;
```

■ **Tip:** ON DELETE CASCADE sabse zyada use hota hai. Jab parent delete ho aur usse related child data bhi automatically hatana ho — jaise user delete karne par uske saare posts bhi delete ho jayein.

5. Practical Queries

5.1 — Sahi Order mein Data Insert Karna

FK ki wajah se parent mein data pehle hona chahiye. Agar customer exist na kare toh uska order nahi daal sakte.

```
-- Step 1: Pehle customer add karo (parent)
INSERT INTO customers (customer_id, name, email)
VALUES (1, 'Rahul Sharma', 'rahul@gmail.com');

-- Step 2: Ab order add karo (child) -- customer_id=1 exist karta hai
INSERT INTO orders (order_id, customer_id, amount)
VALUES (101, 1, 1499.00);
```

X Galat Example: INSERT INTO orders VALUES (102, 999, 500.00) — customer_id=999 exist nahi karta, toh ERROR aayega: *Cannot add or update a child row: a foreign key constraint fails*

Output — customers table:

customer_id	name	email
1	Rahul Sharma	rahul@gmail.com

1 row inserted successfully ✓

Output — orders table:

order_id	customer_id	amount
101	1	1499.00

1 row inserted successfully ✓

5.2 — JOIN ke saath FK Use Karna

```
-- Customer ke naam ke saath unke orders dekhna
SELECT
  c.name      AS customer_name,
  c.email     AS email,
  o.order_id  AS order_id,
  o.amount    AS total_amount
FROM customers c
INNER JOIN orders o ON c.customer_id = o.customer_id;
```

Output:

customer_name	email	order_id	total_amount
Rahul Sharma	rahul@gmail.com	101	1499.00
Priya Singh	priya@gmail.com	102	899.00
Amit Verma	amit@gmail.com	103	2200.00
Priya Singh	priya@gmail.com	104	450.00

4 rows in set — customers aur orders INNER JOIN ka result ✓

5.3 — Existing Table ki FK Constraints Dekhna

```
-- MySQL mein foreign keys check karo
SELECT
  CONSTRAINT_NAME,
  COLUMN_NAME,
  REFERENCED_TABLE_NAME,
  REFERENCED_COLUMN_NAME
FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE
WHERE
  TABLE_NAME = 'orders'
  AND REFERENCED_TABLE_NAME IS NOT NULL;
```

Output:

CONSTRAINT_NAME	COLUMN_NAME	REFERENCED_TABLE E	REFERENCED_COL
fk_customer	customer_id	customers	customer_id

orders table ki FK constraints — 1 row in set ✓

Notes by
Aparna Varshney

6. Self-Referencing Foreign Key

Kabhi kabhi ek table ki row, usi table ki doosri row ko reference karti hai. Isse Self-Referencing ya Recursive Foreign Key kehte hain.

■ **Real-world example:** Company mein ek employee ka manager bhi ek employee hota hai. Toh employees table mein manager_id column, usi table ke emp_id ko reference karta hai.

```
CREATE TABLE employees (
  emp_id      INT          PRIMARY KEY,
  name        VARCHAR(100),
  designation VARCHAR(50),
  manager_id  INT,          -- isi table ka FK!

  FOREIGN KEY (manager_id) REFERENCES employees(emp_id)
  ON DELETE SET NULL
);

-- Data insert karna
INSERT INTO employees VALUES (1, 'Amit (CEO)', 'CEO', NULL); -- CEO ka koi manager nahi
INSERT INTO employees VALUES (2, 'Priya', 'Manager', 1); -- Priya ka manager Amit hai
INSERT INTO employees VALUES (3, 'Ravi', 'Developer', 2); -- Ravi ka manager Priya hai
```

emp_id	name	designation	manager_id
1	Amit (CEO)	CEO	NULL
2	Priya	Manager	1 ← Amit ka ID
3	Ravi	Developer	2 ← Priya ka ID

manager_id → emp_id (same table) — yeh hai Self-Referencing FK

7. Common Errors aur Solutions

■ Cannot add or update a child row

Cause: Child mein aisi value daali jo parent mein exist nahi karti.

Solution: Pehle parent mein data insert karo, phir child mein.

■ Cannot delete or update a parent row

Cause: Parent row delete karne ki koshish ki jab child rows exist karti hain.

Solution: Pehle child rows delete karo, ya ON DELETE CASCADE use karo.

■ Referencing column and referenced column incompatible

Cause: FK aur PK ke data types alag hain (e.g., INT vs VARCHAR).

Solution: Dono columns ka data type aur size bilkul same rakho.

■ FK constraint on non-existent column

Cause: Galat column naam likha FK mein.

Solution: Table ka DESCRIBE karke column naam check karo.

8. Best Practices

Best Practice	Kyun?
Constraint ko naam do (fk_table_column)	Debug karna aur drop karna aasaan hota hai
FK columns mein INDEX lagao	JOIN queries fast ho jaati hain
Parent table pehle banao	Dependency order maintain hota hai
FK column aur referenced column same type rakho	Type mismatch error se bachta hai
ON DELETE behavior soch samajh kar choose karo	Accidental data loss se bachata hai
NULL allow karna sochkar decide karo	NULL FK = optional relationship

9. Quick Reference Card

Task	Syntax
Naya table mein FK	FOREIGN KEY (col) REFERENCES parent(col)
CONSTRAINT naam ke saath	CONSTRAINT fk_name FOREIGN KEY ...
Existing table mein FK add	ALTER TABLE t ADD CONSTRAINT ... FOREIGN KEY ...
FK remove karna (MySQL)	ALTER TABLE t DROP FOREIGN KEY fk_name
FK remove karna (PostgreSQL)	ALTER TABLE t DROP CONSTRAINT fk_name
Auto delete children	ON DELETE CASCADE
Delete rokna	ON DELETE RESTRICT
NULL set karna	ON DELETE SET NULL
FKs list dekhna (MySQL)	SELECT ... FROM INFORMATION_SCHEMA.KEY_COLUMN_USAGE

10. Practice Exercises

Exercise 1 — Basic

Ek School database banao jisme students aur courses tables hon. Ek teen table enrollments banao jo students aur courses dono se FK rakhti ho.

Exercise 2 — CASCADE

Ek Blog database banao jisme users aur posts tables hon. Aise set karo ki user delete hone par uske saare posts bhi delete ho jaayein (CASCADE).

Exercise 3 — Self-Referencing

Ek categories table banao jisme ek category ki parent_category bhi same table se ho. (e.g., Electronics > Mobile Phones > Smartphones)

Exercise 4 — ALTER TABLE

Pehle products table banao bina FK ke. Phir ALTER TABLE se ek supplier_id FK add karo jo suppliers table ko reference kare.

11. Exercise Solutions

■ Pehle khud try karo, phir yahan check karo! Solutions dekhne se pehle apna code likhna zaroori hai.

Exercise 1 — School Database (Basic FK)

```
-- Step 1: students table (parent)
CREATE TABLE students (
  student_id INT          PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  roll_no     VARCHAR(20)
);

-- Step 2: courses table (parent)
CREATE TABLE courses (
  course_id INT          PRIMARY KEY,
  course_name VARCHAR(100),
  credits    INT
);

-- Step 3: enrollments table (child - dono se FK)
CREATE TABLE enrollments (
  enroll_id INT PRIMARY KEY,
  student_id INT NOT NULL,
  course_id  INT NOT NULL,
  enroll_date DATE,

  FOREIGN KEY (student_id) REFERENCES students(student_id),
  FOREIGN KEY (course_id)  REFERENCES courses(course_id)
);
```

Sample Output — enrollments table:

enroll_id	student_id	course_id	enroll_date
1	101 → students	201 → courses	2024-01-15
2	102 → students	202 → courses	2024-01-16
3	101 → students	203 → courses	2024-01-17

ek student kai courses mein enroll ho sakta hai — Many-to-Many relationship ✓

Exercise 2 — Blog Database (ON DELETE CASCADE)

```

CREATE TABLE users (
  user_id INT PRIMARY KEY,
  username VARCHAR(50) NOT NULL,
  email VARCHAR(100)
);

CREATE TABLE posts (
  post_id INT PRIMARY KEY,
  user_id INT NOT NULL,
  title VARCHAR(200),
  content TEXT,

  FOREIGN KEY (user_id) REFERENCES users(user_id)
  ON DELETE CASCADE -- user delete → posts bhi delete!
);

-- Test: user delete karo
DELETE FROM users WHERE user_id = 1;
-- Result: user_id=1 ke saare posts bhi automatically delete ho gaye!

```

CASCADE ka behavior — Before aur After Delete:

Pehle (Before DELETE)			Baad mein (After DELETE user_id=1)		
post_id	user_id	title	post_id	user_id	title
1	1	My First Post	3	2	Python Guide
2	1	SQL Tutorial	(post1 deleted)		
3	2	Python Guide	(post2 deleted)		

user_id=1 delete hone par uske dono posts bhi automatically delete ho gaye ✓

Exercise 3 — Self-Referencing (Categories)

```

CREATE TABLE categories (
  cat_id INT PRIMARY KEY,
  cat_name VARCHAR(100) NOT NULL,
  parent_cat_id INT DEFAULT NULL, -- top-level ka NULL

  FOREIGN KEY (parent_cat_id) REFERENCES categories(cat_id)
  ON DELETE SET NULL
);

-- Data insert karna (hierarchy banao)
INSERT INTO categories VALUES (1, 'Electronics', NULL);
INSERT INTO categories VALUES (2, 'Mobile Phones', 1); -- Electronics ke under
INSERT INTO categories VALUES (3, 'Smartphones', 2); -- Mobile Phones ke under

```

Output — categories table (hierarchy):

cat_id	cat_name	parent_cat_id	Level
1	Electronics	NULL	Level 1 (Root)
2	Mobile Phones	1	Level 2
3	Smartphones	2	Level 3

Electronics → Mobile Phones → Smartphones (3-level hierarchy) ✓

Exercise 4 — ALTER TABLE se FK Add Karna

```
-- Step 1: suppliers table banao (parent)
CREATE TABLE suppliers (
  supplier_id INT PRIMARY KEY,
  supplier_name VARCHAR(100),
  contact VARCHAR(15)
);

-- Step 2: products table banao BINA FK ke
CREATE TABLE products (
  product_id INT PRIMARY KEY,
  product_name VARCHAR(100),
  price DECIMAL(10,2),
  supplier_id INT
);

-- Step 3: ALTER TABLE se FK add karo
ALTER TABLE products
ADD CONSTRAINT fk_supplier
  FOREIGN KEY (supplier_id) REFERENCES suppliers(supplier_id)
  ON DELETE SET NULL;

-- Verify karo ki FK add hua
SHOW CREATE TABLE products;
```

Output — SHOW CREATE TABLE products:

```
-- Output mein ye line dikhegi:
CONSTRAINT fk_supplier FOREIGN KEY (supplier_id)
  REFERENCES suppliers (supplier_id)
  ON DELETE SET NULL
```

■ FK successfully add ho gaya! Ab products mein sirf wahi supplier_id daal sakte ho jo suppliers table mein exist karta ho. ✓